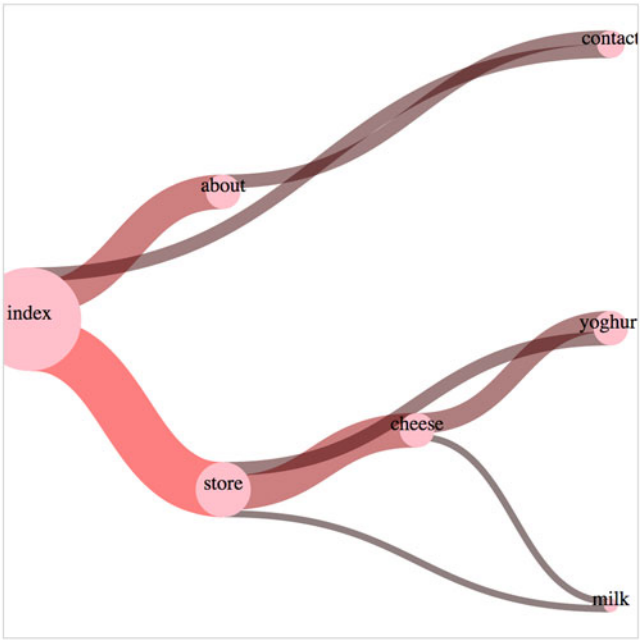


Figure 5.25 A squid-like Sankey diagram

hard time adapting the layout data to any element other than a `<path>`, but the nodes don't need to be `<rect>` elements. If we adjust our code, we can easily make nodes that are circles:

```
  sankey.nodeWidth(1);

d3.selectAll(".node").append("circle")
    .attr("height", function(d) { return d.dy; })
    .attr("r", function(d) { return d.dy / 2; })
    .attr("cy", function(d) { return d.dy / 2; })
    .style("fill", "pink")
    .style("stroke", "gray");
```

Don't shy away from experimenting with tweaks to traditional charting methods. Using circles instead of rectangles, like in figure 5.25, may seem frivolous, but it may be a better fit visually, or it may distinguish your Sankey from all the boring sharp-edged Sankeys out there. In the same vein, don't be afraid of leveraging D3's capacity for information visualization to teach yourself how a layout works. You'll remember that `d3.layout.sankey` has a `layout()` function, and you might discover the operation of that function by reading the code. But there's another way for you to see how this function works: by using transitions and creating a function that updates the `.layout()` property dynamically, you can see what this function does to the chart graphically.

VISUALIZING ALGORITHMS Although you may think of data visualization as all the graphics in this book, it's also simultaneously a graphical representation